Quine-McCluskey Method

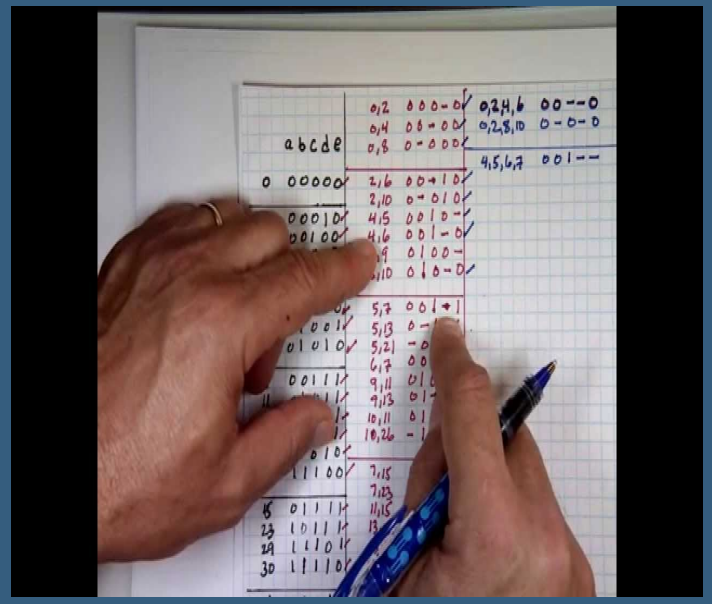
Digital System Fundamentals

Sorayut Glomglome

Learning Outcome

- Use the Quine-McCluskey (Q-M) method to simplify Boolean expressions.

CONTENTS



1. Example Applications

2. Universal Gates

3. Quine-McCluskey Method

4. Quine-McCluskey Method with *Don't Care*

5. Implementing Combinational Logic Circuit

1. Example Applications

A Burglar Alarm

A burglar alarm for a bank is designed so that it senses four input signal lines. Line A is from the secret control switch, line B is from a pressure sensor under a steel safe in a locked closet, line C is from a battery-powered clock, and line D is connected to a switch on the locked closet door. The following conditions produce a logic 1 voltage on each line:

A : The control switch is closed.

B : The safe is in its normal position in the closet.

C : The clock is between 1000 and 1400 hours.

D : The closet door is closed.

Write the equations of the control logic for the burglar alarm that produces a logic 1 (rings a bell) when the safe is moved and the control switch is closed, or when the closet is opened after banking hours, or when the closet is opened with the control switch open.

The statement "when the safe is moved and the control switch is closed" is represented by $A\bar{B}$. "When the closet is opened after banking hours" is represented by $\bar{C}\bar{D}$. "When the closet is opened with the control switch open" is represented by $\bar{A}\bar{D}$. Therefore, the logic equation for the burglar alarm is

$$f(A, B, C, D) = A\bar{B} + \bar{C}\bar{D} + \bar{A}\bar{D}$$

A Restaurant Voting

John and Jane Doe have two children, Joe and Sue. When eating out they will go to a restaurant that serves only hamburgers or one that serves only chicken. Before going out, the family votes to decide on the restaurant. The majority wins, except when Mom and Dad agree, and in that case they win. Any other tie votes produce a trip to the chicken restaurant. We wish to design a logic circuit that will automatically select the restaurant when everyone votes.

If we let a 1 represent a vote cast for hamburgers and a 0 represent a vote cast for chicken, the truth table for the voting circuit is given in Table 2.9. The logic function is

TABLE 2.9 TRUTH TABLE FOR DOE FAMILY VOTER

John A	Jane B	Joe C	Sue D	Vote for Hamburger f
0	0	0	0	
0	0	0	1	
0	0	1	0	
0	0	1	1	
0	1	0	0	
0	1	0	1	
0	1	1	0	
0	1	1	1	
1	0	0	0	
1	0	0	1	
1	0	1	0	
1	0	1	1	
1	1	0	0	
1	1	0	1	
1	1	1	0	
1	1	1	1	

A Restaurant Voting

John and Jane Doe have two children, Joe and Sue. When eating out they will go to a restaurant that serves only hamburgers or one that serves only chicken. Before going out, the family votes to decide on the restaurant. The majority wins, except when Mom and Dad agree, and in that case they win. Any other tie votes produce a trip to the chicken restaurant. We wish to design a logic circuit that will automatically select the restaurant when everyone votes.

If we let a 1 represent a vote cast for hamburgers and a 0 represent a vote cast for chicken, the truth table for the voting circuit is given in Table 2.9. The logic function is

$$
\begin{aligned}
f(A, B, C, D) &= \bar{A}BCD + A\bar{B}CD + AB\bar{C}\bar{D} \\
&\quad + AB\bar{C}D + ABC\bar{D} + ABCD \\
&= \bar{A}BCD + A\bar{B}CD + AB \\
&= AB + ACD + \bar{A}BCD \\
&= AB + ACD + BCD
\end{aligned}
$$

TABLE 2.9 TRUTH TABLE FOR DOE FAMILY VOTER

John A	Jane B	Joe C	Sue D	Vote for Hamburger f
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

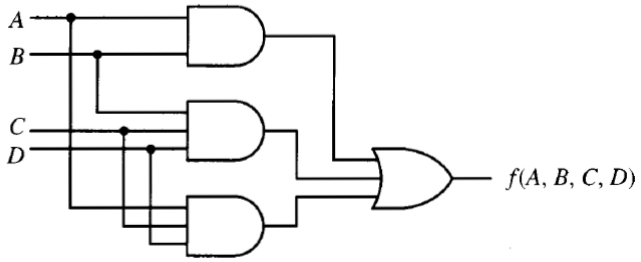
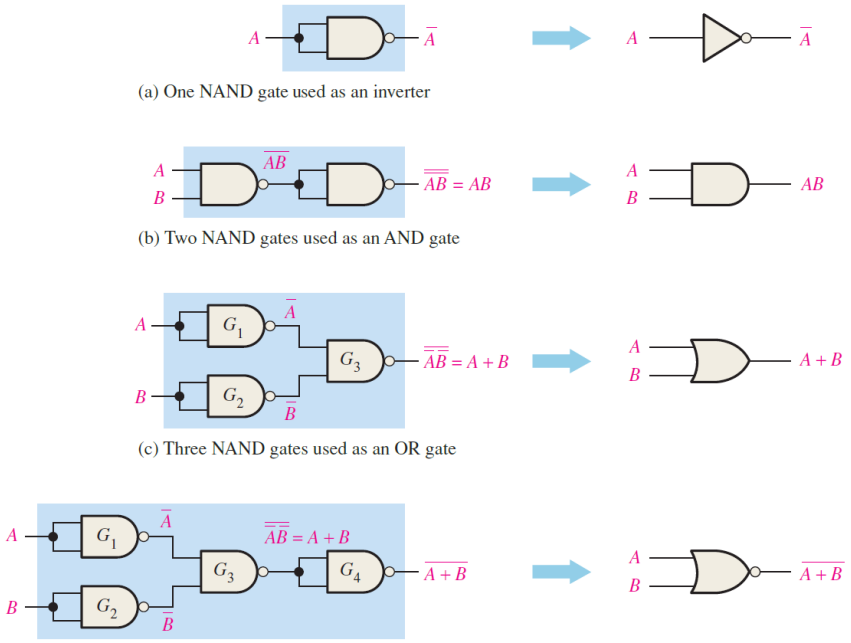


Figure 2.34 Logic circuit for restaurant voter example.

2. Univrsal Gates

Universal NAND Gates

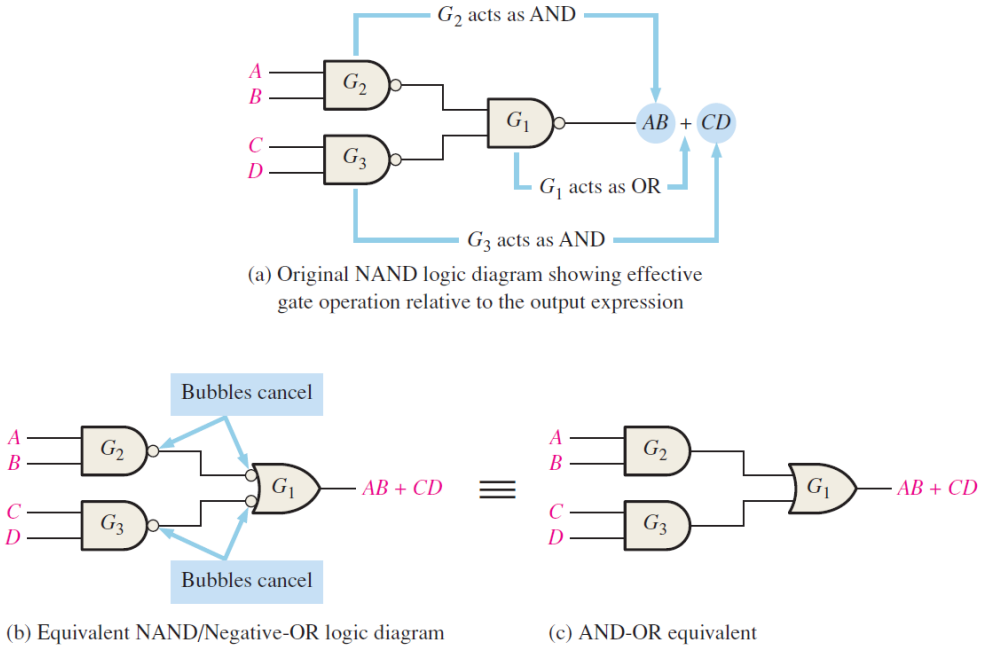


(a) One NAND gate used as an inverter

(b) Two NAND gates used as an AND gate

(c) Three NAND gates used as an OR gate

(d) Four NAND gates used as a NOR gate



(a) Original NAND logic diagram showing effective gate operation relative to the output expression

(b) Equivalent NAND/Negative-OR logic diagram

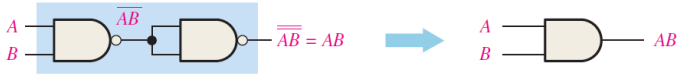
(c) AND-OR equivalent

FIGURE 5–21 Development of the AND-OR equivalent of the circuit in Figure 5–20.

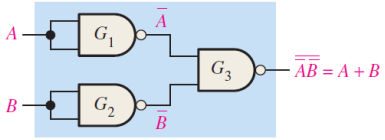
Universal NAND Gates

2.33 Derive the logic equation and circuit diagram for a circuit with three inputs A, B, and C. The output is to be high only when exactly one of the three inputs is high. Use only NAND gates in the design.

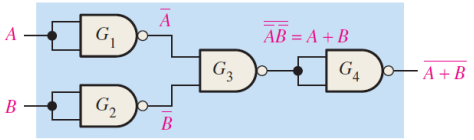
(a) One NAND gate used as an inverter



(b) Two NAND gates used as an AND gate

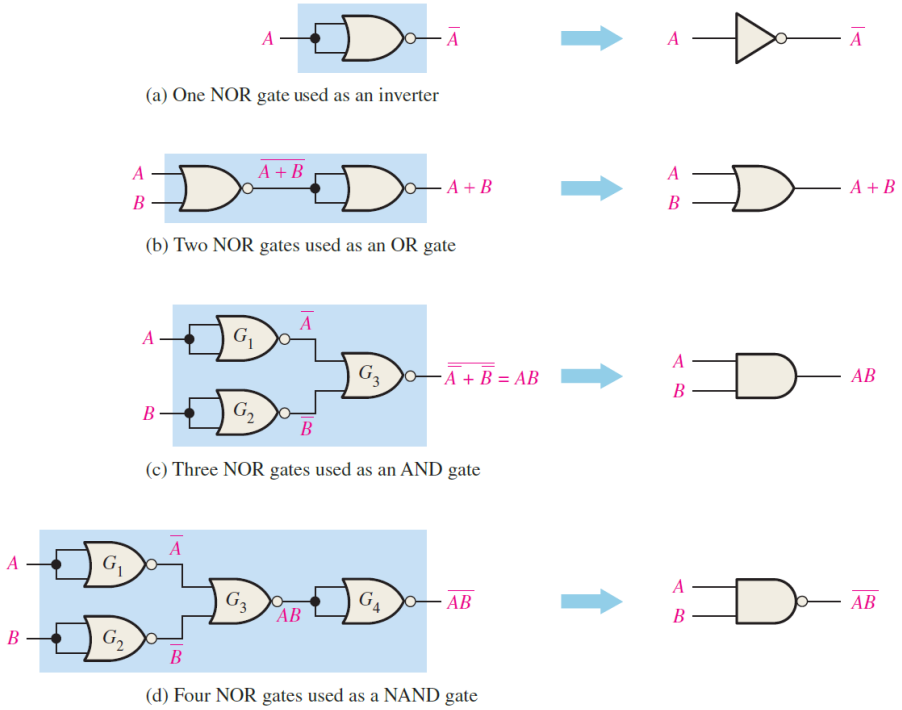


(c) Three NAND gates used as an OR gate



(d) Four NAND gates used as a NOR gate

Universal NOR Gates



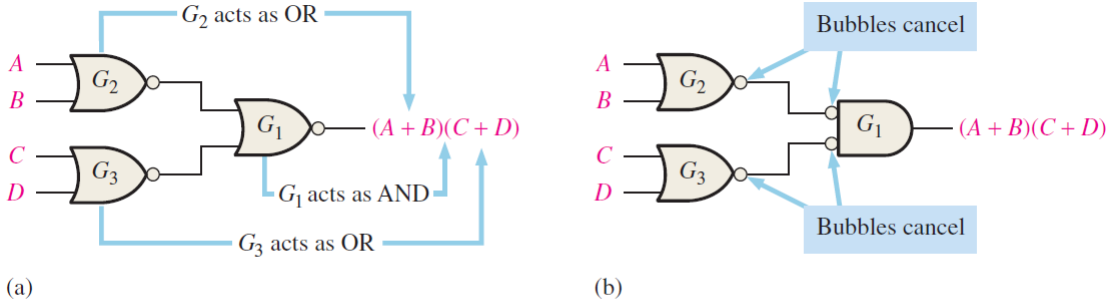
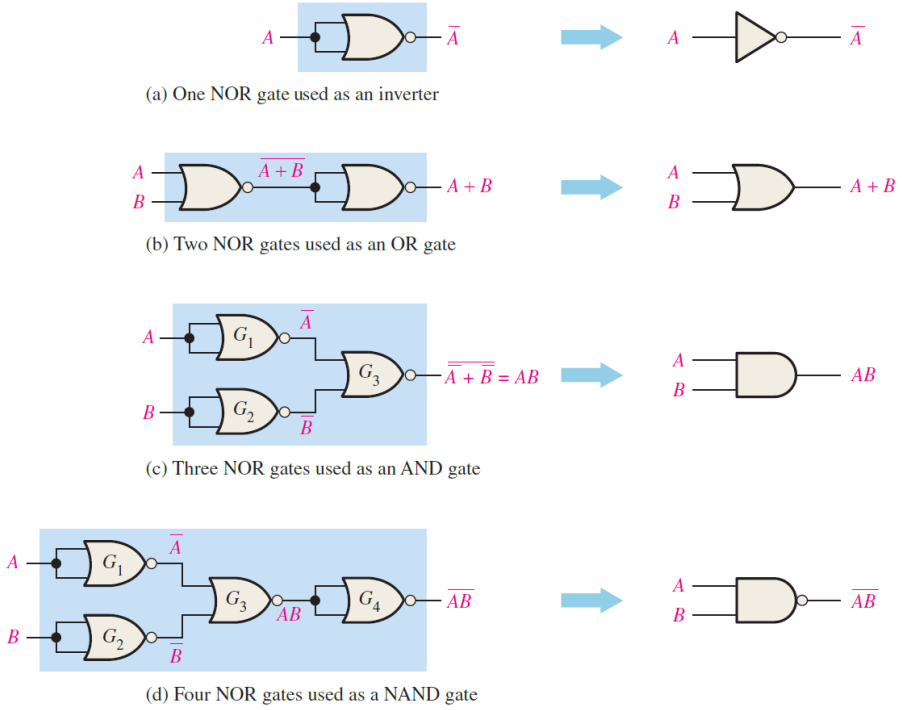


FIGURE 5–27

Universal NOR Gates

2.34 We wish to design a logic circuit with four inputs A, B, C, and D. The output is to be high only when a majority of the inputs is high. Draw the final circuit using only NOR gates.



(a) One NOR gate used as an inverter

(b) Two NOR gates used as an OR gate

(c) Three NOR gates used as an AND gate

(d) Four NOR gates used as a NAND gate

3. Quine-McCluskey Method

Quine-McCluskey (Q-M) Method

- A tabular approach to Boolean function/expression minimization.
- Two advantages over the K-Map:
  - Systematic method, less dependent on the designer's ability.
  - Handling a large number of variables .
- Performs a linear search over the minterms to find all combinations of logically adjacent minterms.

Step 1

- List in a column all the minterms of the function to be minimized in their binary representation.
- Partition them into groups according to the number of 1 bits in their binary representations.
- This partitioning simplifies identification of logically adjacent minterms since two minterms must differ in exactly one literal, and therefore the binary representation of one minterm must have either one more or one fewer 1 bit than the other.

Step 2

- Perform an exhaustive search between neighboring groups for adjacent minterms and combine them into a column of (n - 1)-variable implicants, checking off each minterm that is combined.
- The binary representation of each new implicant contains a dash in the position of the eliminated variable.
- Repeat for each column, combining (n - 1)-varible implicants into (n - 2)-variable implicants, and so on, until no further implicants can be combined.
- Any term not checked off represents a prime implicant of the function, since it is not covered by a larger implicant.
- The final result is a list of prime implicants.

Step 3

- Construct a prime implicant chart that lists minterms along the horizontal and prime implicants along the vertical, with an x entry placed wherever a certain prime implicant (row) covers a given minterm (column).

Step 4

- Select a minimum number of prime implicants that cover all the minterms.

Example

Let us use the Q–M technique to minimize the function

$$f(A,B,C,D) = \sum m(2,4,6,8,9,10,12,13,15)$$

The K-map for this example is shown in Fig. 3.32, and the reader is encouraged to try his or her hand at obtaining a minimal function via the map method.

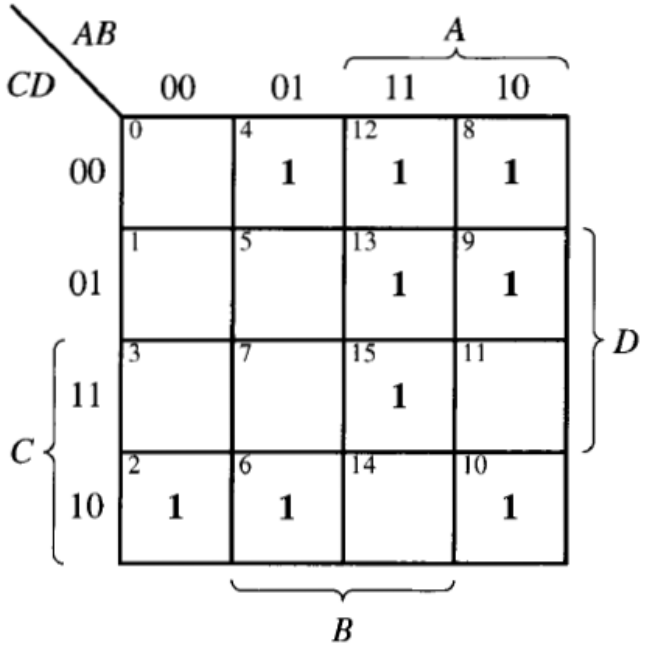


Figure 3.32 K-map for Example 3.30.

Step 1. To begin the Q–M minimization technique, the minterms are grouped according to the number of ones in the binary representation of the minterm number. This grouping of terms is illustrated in the following table:

Minterms	$ABCD$	
2	0010	
4	0100	Group 1 (a single 1)
8	1000	
6	0110	
9	1001	Group 2 (two 1's)
10	1010	
12	1100	
13	1101	Group 3 (three 1's)
15	1111	Group 4 (four 1's)

Step 2. Once this table has been formed, an exhaustive search for all combinations of logically adjacent terms is initiated. The method of performing this

List 1			List 2			List 3		
Minterm	$ABCD$		Minterms	$ABCD$		Minterms	$ABCD$	
2	0010	√	2, 6	0-10	PI_2	8, 9, 12, 13	1-0-	PI_1
4	0100	√	2, 10	-010	PI_3			
8	1000	√	4, 6	01-0	PI_4			
6	0110	√	4, 12	-100	PI_5			
9	1001	√	8, 9	100-	√			
10	1010	√	8, 10	10-0	PI_6			
12	1100	√	8, 12	1-00	√			
13	1101	√	9, 13	1-01	√			
15	1111	√	12, 13	110-	√			
			13, 15	11-1	PI_7			

$$\text{minterm } 2 = \bar{A}\bar{B}C\bar{D},$$

$$\text{minterm } 6 = \bar{A}BC\bar{D}$$

$$\bar{A}\bar{B}C\bar{D} + \bar{A}BC\bar{D} = \bar{A}C\bar{D} \Rightarrow 0-10$$

Step 3. To determine the smallest number of prime implicants required to realize the function we form a prime implicant chart as follows:

	2	4	6	√ 8	√ 9	10	√ 12	√ 13	√ 15
** PI_1				×	⊗		×	×	
PI_2	×		×						
PI_3	×					×			
PI_4		×	×						
PI_5		×					×		
PI_6				×		×			
** PI_7								×	⊗

The double horizontal line through the chart between PI_1 and PI_2 is used to separate prime implicants that contain different numbers of literals.

Step 4. An examination of the minterm columns in the prime implicant chart indicates that minterms 9 and 15 are each covered by only one prime implicant (shown circled). Therefore, prime implicants 1 and 7 must be chosen, and hence they are essential prime implicants (as indicated by the double asterisks). Note that in choosing these two prime implicants we have *also* covered minterms 8, 12, and 13. All five of the covered minterms are checked in the table; the checks are placed above the minterm numbers.

The problem now remaining is that of selecting as few additional (nonessential) prime implicants as are necessary to cover the minterms 2, 4, 6, and 10. In general, this is accomplished by forming a reduced prime implicant chart. This reduced chart is shown next; note that the chart contains only the minterms that remain to be covered and the remaining prime implicant candidates for inclusion in the cover.

	2	4	6	√ 8	√ 9	10	√ 12	√ 13	√ 15
** PI_1				×	⊗		×	×	
PI_2	×		×						
PI_3	×					×			
PI_4		×	×						
PI_5		×					×		
PI_6				×		×			
** PI_7								×	⊗

	√ 2	√ 4	√ 6	√ 10
PI_2	×		×	
*PI_3	×			×
*PI_4		×	×	
PI_5		×		
PI_6				×

Which PIs should we select? Prime implicants PI_5 and PI_6 are obviously bad choices because they cover only one minterm, and that minterm is also covered by another PI that covers two minterms. Notice that the minterms 2, 4, 6, and 10 can be most efficiently covered (with the minimum number of prime implicants) by choosing PI_3 and PI_4. The single asterisk indicates our selection, and the checks above all the remaining minterms mean we have generated a complete cover. Therefore, a minimal realization of the original function would be

$$
\begin{aligned}
f(A, B, C, D) &= PI_1 + PI_3 + PI_4 + PI_7 \\
&= 1\text{-}0\text{-} + \text{-}010 + 01\text{-}0 + 11\text{-}1 \\
&= A\bar{C} + \bar{B}C\bar{D} + \bar{A}B\bar{D} + ABD
\end{aligned}
$$

List 1			List 2			List 3		
Minterm	$ABCD$		Minterms	$ABCD$		Minterms	$ABCD$	
2	0010	√	2, 6	0-10	PI_2	8, 9, 12, 13	1-0-	PI_1
4	0100	√	2, 10	-010	PI_3			
8	1000	√	4, 6	01-0	PI_4			
6	0110	√	4, 12	-100	PI_5			
9	1001	√	8, 9	100-	√			
10	1010	√	8, 10	10-0	PI_6			
12	1100	√	8, 12	1-00	√			
13	1101	√	9, 13	1-01	√			
15	1111	√	12, 13	110-	√			
			13, 15	11-1	PI_7			

The corresponding groupings of the minterms on the K-map are shown in Fig. 3.33.

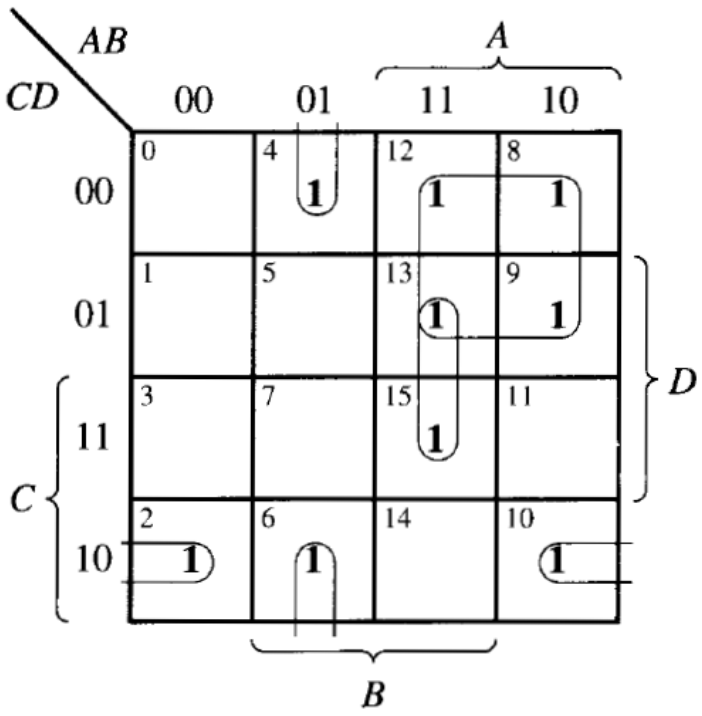


Figure 3.33 Grouping of terms.

4. Quine-McCluskey Method with *Don't Care*

3.9.2 Incompletely Specified Functions

The minimization of functions involving don't-cares proceeds exactly as shown in the preceding example with one important exception, which will be demonstrated by the next example.

We want to use the Q–M approach to minimize the function

$$f(A,B,C,D,E) = m(2,3,7,10,12,15,27) + d(5,18,19,21,23)$$

Following the procedure demonstrated in the preceding example, all the minterms *and don't-cares* are listed in the minimizing table and combined in the manner previously illustrated. The results of this procedure are shown in the following table:

$$f(A,B,C,D,E) = m(2,3,7,10,12,15,27) + d(5,18,19,21,23)$$

Put *don't care* in the List.

Do the same process.

List 1			List 2			List 3		
Minterm	$ABCDE$		Minterms	$ABCDE$		Minterms	$ABCDE$	
2	00010	√	2, 3	0001-	√	2, 3, 18, 19	-001-	PI_1
3	00011	√	2, 10	0-010	PI_4	3, 7, 19, 23	-0-11	PI_2
5	00101	√	2, 18	-0010	√	5, 7, 21, 23	-01-1	PI_3
10	01010	√	3, 7	00-11	√			
12	01100	PI_7	3, 19	-0011	√			
18	10010	√	5, 7	001-1	√			
7	00111	√	5, 21	-0101	√			
19	10011	√	18, 19	1001-	√			
21	10101	√	7, 15	0-111	PI_5			
15	01111	√	7, 23	-0111	√			
23	10111	√	19, 23	10-11	√			
27	11011	√	19, 27	1-011	PI_6			
			21, 23	101-1	√			

Do not list *don't care* in PI chart

	√ 2	3	√ 7	√ 10	√ 12	√ 15	√ 27
PI_1	×	×					
PI_2		×	×				
PI_3			×				
** PI_4	×			⊗			
** PI_5			×			⊗	
** PI_6							⊗
** PI_7					⊗		

$$f(A, B, C, D, E) = PI_1 + PI_4 + PI_5 + PI_6 + PI_7$$

$$f(A, B, C, D, E) = PI_2 + PI_4 + PI_5 + PI_6 + PI_7$$

In terms of the variables,

$$f(A, B, C, D, E) = \bar{B}\bar{C}D + \bar{A}\bar{C}D\bar{E} + \bar{A}CDE + A\bar{C}DE + \bar{A}BC\bar{D}\bar{E}$$

or

$$f(A, B, C, D, E) = \bar{B}DE + \bar{A}\bar{C}D\bar{E} + \bar{A}CDE + A\bar{C}DE + \bar{A}BC\bar{D}\bar{E}$$

Problems / Practice 5.1

3.56 Use the Quine–McCluskey method to minimize the following functions.

(a) $f(A, B, C, D) = \sum m(0, 2, 3, 5, 7, 11, 12, 14, 15)$

(b) $f(A, B, C, D) = \sum m(0, 1, 6, 8, 9, 13, 14, 15)$

3.57 Use the Quine–McCluskey method to minimize the following functions.

(a) $f(A, B, C, D) = \sum m(1, 4, 5, 6, 8, 9, 10, 12, 14)$

(b) $f(A, B, C, D) = \sum m(4, 5, 6, 8, 11, 13, 15)$

3.58 Minimize the following functions using the Quine–McCluskey method.

(a) $f(A, B, C, D) = \sum m(1, 3, 6, 7, 8, 9, 12, 14)$

(b) $f(A, B, C, D) = \sum m(0, 2, 4, 5, 10, 11, 13, 15)$

3.59 Use the Quine–McCluskey method to minimize the following functions with don't-cares.

(a) $f(A, B, C, D) = \sum m(1, 6, 7, 9, 12) + d(8, 11, 15)$

(b) $f(A, B, C, D) = \sum m(7, 8, 13, 15) + d(3, 4, 10, 14)$

3.60 Minimize the following functions with don't-cares using the Quine–McCluskey method.

(a) $f(A, B, C, D) = \sum m(5, 7, 11, 12, 27, 29) + d(14, 20, 21, 22, 23)$

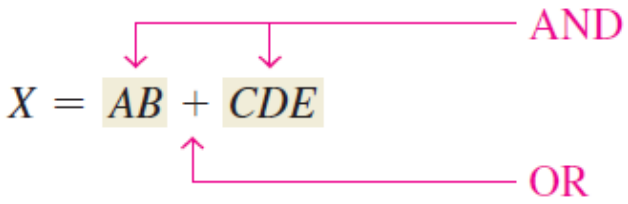
(b) $f(A, B, C, D) = \sum m(1, 4, 6, 9, 14, 17, 22, 27, 28) + d(12, 15, 20, 30, 31)$

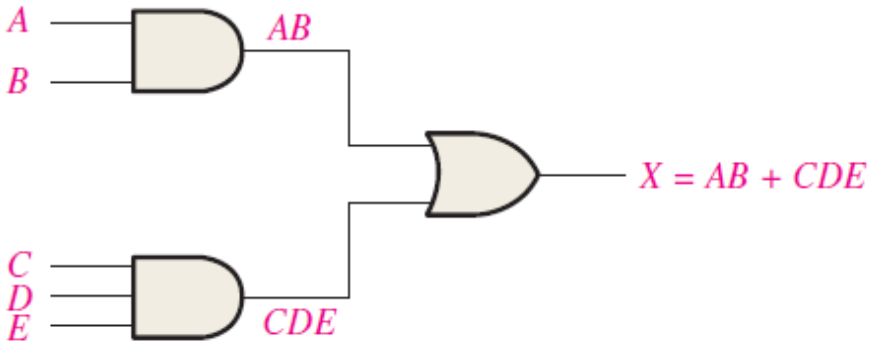
5. Implementing Combinational Logic Circuit

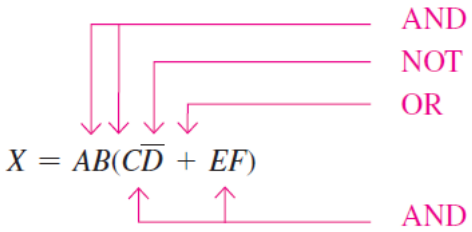
Implementing Combinational Logic Circuit

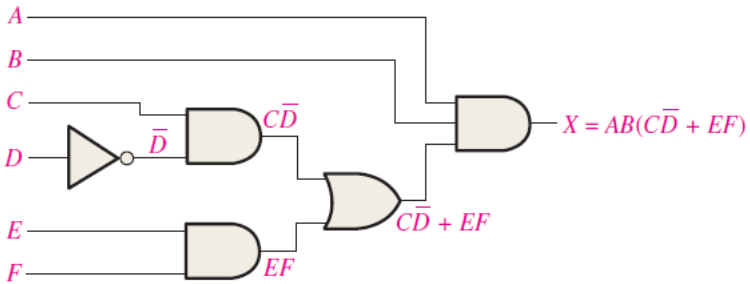
1. Implement a logic circuit from a Boolean expression
2. Implement a logic circuit from a truth table
3. Minimize a logic circuit

Implement a logic circuit from a Boolean expression







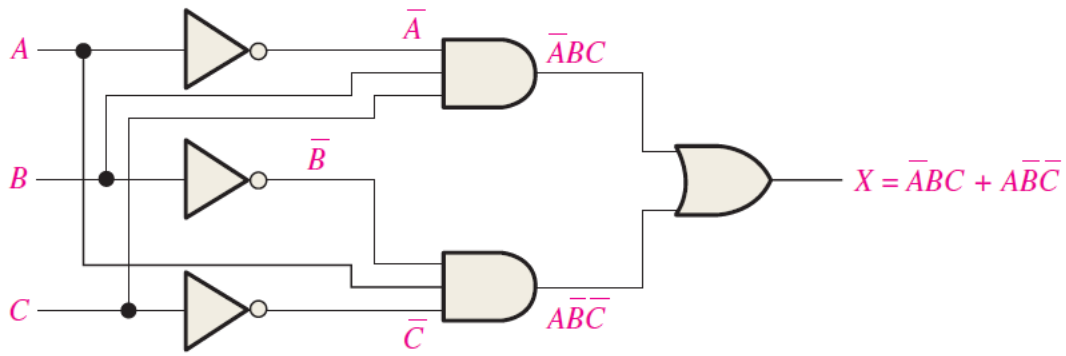


From a Truth Table to a Logic Circuit

TABLE 5–3

Inputs			Output	
A	B	C	X	Product Term
0	0	0	0	
0	0	1	0	
0	1	0	0	
0	1	1	1	$\overline{A}BC$
1	0	0	1	$A\overline{B}\,\overline{C}$
1	0	1	0	
1	1	0	0	
1	1	1	0	

$$X = \overline{A}BC + A\overline{B}\,\overline{C}$$

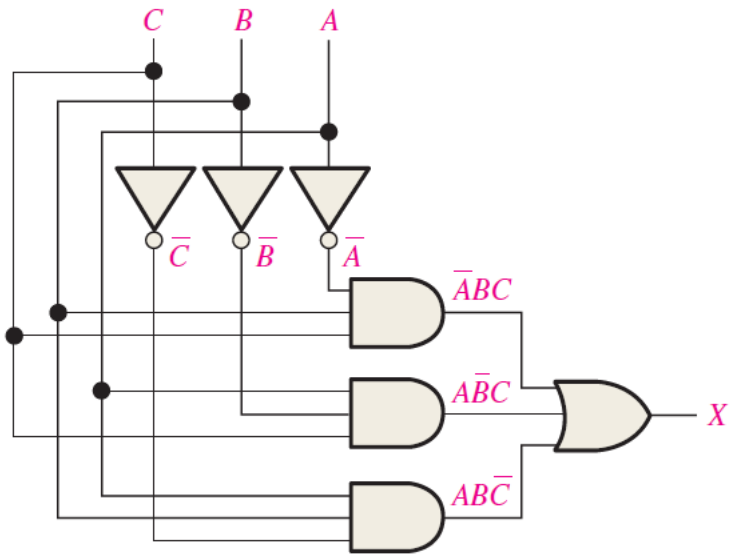


From a Truth Table to a Logic Circuit

Design a logic circuit to implement the operation specified in the truth table of Table 5–4.

TABLE 5–4

Inputs			Output	
A	B	C	X	Product Term
0	0	0	0	
0	0	1	0	
0	1	0	0	
0	1	1	1	$\overline{A}BC$
1	0	0	0	
1	0	1	1	$A\overline{B}C$
1	1	0	1	$AB\overline{C}$
1	1	1	0	



Solution

Notice that $X = 1$ for only three of the input conditions. Therefore, the logic expression is

$$X = \overline{A}BC + A\overline{B}C + AB\overline{C}$$

From a Truth Table to a Logic Circuit

Develop a logic circuit with four input variables that will only produce a 1 output when exactly three input variables are 1s.

Solution

Out of sixteen possible combinations of four variables, the combinations in which there are exactly three 1s are listed in Table 5–5, along with the corresponding product term for each.

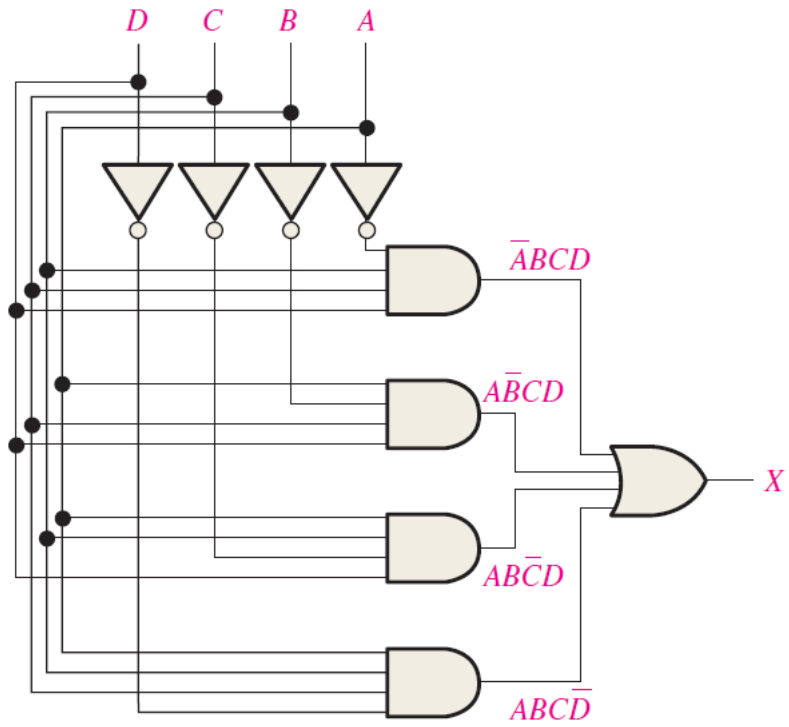
TABLE 5–5

A	B	C	D	Product Term
0	1	1	1	$\overline{A}BCD$
1	0	1	1	$A\overline{B}CD$
1	1	0	1	$AB\overline{C}D$
1	1	1	0	$ABC\overline{D}$

The product terms are ORed to get the following expression:

$$X = \overline{A}BCD + A\overline{B}CD + AB\overline{C}D + ABC\overline{D}$$

This expression is implemented in Figure 5–13 with AND-OR logic.



Minimize a logic circuit

Reduce the combinational logic circuit in Figure 5–14 to a minimum form.

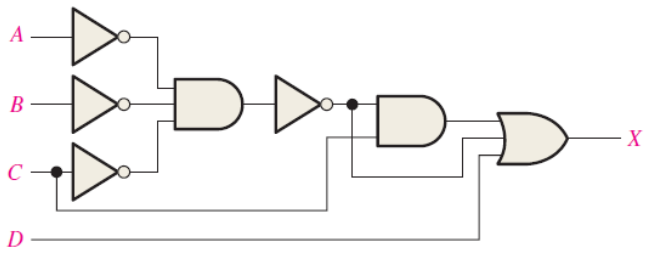


FIGURE 5–14
Open file F05-14 to verify that this circuit is equivalent to the gate in Figure 5–15.

MultiSim

Solution

The expression for the output of the circuit is

$$X = (\overline{\overline{A}\,\overline{B}\,\overline{C}})C + \overline{\overline{A}\,\overline{B}\,\overline{C}} + D$$

Applying DeMorgan's theorem and Boolean algebra,

$$\begin{aligned} X &= (\overline{\overline{A}} + \overline{\overline{B}} + \overline{\overline{C}})C + \overline{\overline{A}} + \overline{\overline{B}} + \overline{\overline{C}} + D \\ &= AC + BC + CC + A + B + C + D \\ &= AC + BC + C + A + B + \cancel{C} + D \\ &= C(A + B + 1) + A + B + D \\ X &= A + B + C + D \end{aligned}$$

The simplified circuit is a 4-input OR gate as shown in Figure 5–15.

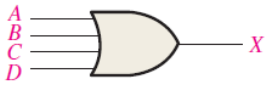


FIGURE 5–15

Minimize a logic circuit

Minimize the combinational logic circuit in Figure 5–16. Inverters for the complemented variables are not shown.

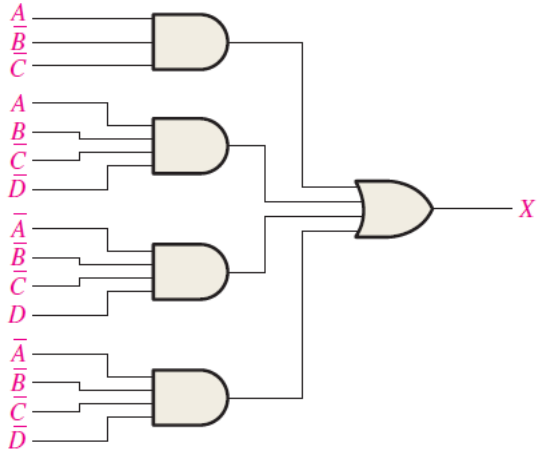


FIGURE 5–16

Solution

The output expression is

$$X = A\overline{B}\,\overline{C} + AB\overline{C}\,\overline{D} + \overline{A}\,\overline{B}\,\overline{C}D + \overline{A}\,\overline{B}\,\overline{C}\,\overline{D}$$

Expanding the first term to include the missing variables D and $\overline{D}$,

$$X = A\overline{B}\,\overline{C}(D + \overline{D}) + AB\overline{C}\,\overline{D} + \overline{A}\,\overline{B}\,\overline{C}D + \overline{A}\,\overline{B}\,\overline{C}\,\overline{D}$$
$$= A\overline{B}\,\overline{C}D + A\overline{B}\,\overline{C}\,\overline{D} + AB\overline{C}\,\overline{D} + \overline{A}\,\overline{B}\,\overline{C}D + \overline{A}\,\overline{B}\,\overline{C}\,\overline{D}$$

This expanded SOP expression is mapped and simplified on the Karnaugh map in Figure 5–17(a). The simplified implementation is shown in part (b). Inverters are not shown.

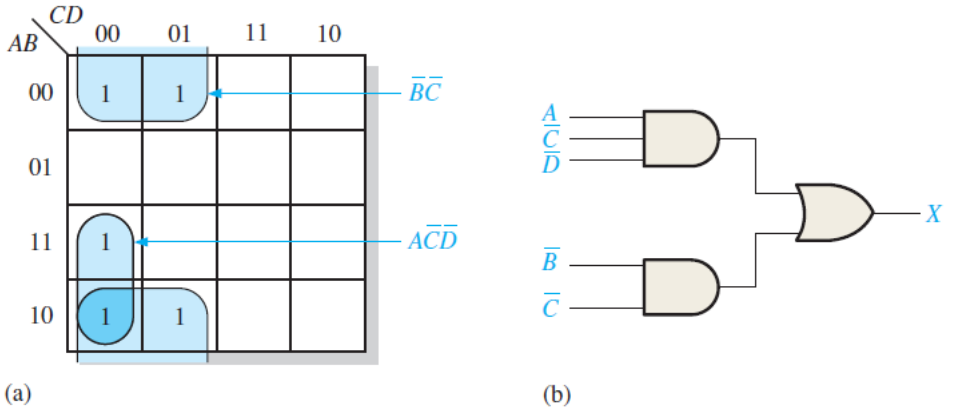


FIGURE 5–17

THANK YOU